Ulster University

jQuery – Part Two

cp.shewell@ulster.ac.uk

Show and Hide Effects

- You can show and hide HTML elements using the jQuery show() and hide() methods.
- The hide() method simply sets the inline style display: none for the selected elements.
- Conversely, the show() method restores the display properties of the matched set of elements to whatever they initially were—typically block, inline, or inline-block—before the inline style display: none was applied to them.

```
<script>
$(document).ready(function(){
    // Hide displayed paragraphs
    $(".hide-btn").click(function(){
        $("p").hide();
    });

    // Show hidden paragraphs
    $(".show-btn").click(function(){
        $("p").show();
    });
});
</script>
```

Show and Hide Effects

- You can op parameter for making the eriod of time.
- Durations c ring 'slow' or 'fast', or in a er values indicate slo

```
<script>
$(document).ready(function(){
    // Hide displayed paragraphs with different speeds
    $(".hide-btn").click(function(){
        $("p.normal").hide();
        $("p.fast").hide("fast");
        $("p.slow").hide("slow");
        $("p.very-fast").hide(50);
        $("p.very-slow").hide(2000);
    });

    // Show hidden paragraphs with different speeds
    $(".show-btn").click(function(){
        $("p.normal").show();
        $("p.fast").show("fast");
        $("p.slow").show("slow");
        $("p.very-fast").show(50);
        $("p.very-slow").show(2000);
    });
});
</script>
```

Show and Hide Effects

- You can also specify a callback function to be executed after the show() or hide() method completes.

```
<script>
$(document).ready(function(){
    // Display alert message after hiding paragraphs
    $(".hide-btn").click(function(){
        $("p").hide("slow", function(){
            // Code to be executed
            alert("The hide effect is completed.");
        });
    });

    // Display alert message after showing paragraphs
    $(".show-btn").click(function(){
        $("p").show("slow", function(){
            // Code to be executed
            alert("The show effect is completed.");
        });
    });
});
</script>
```

jQuery toggle()

- The jQuery toggle() method show or hide the elements in such a way that if the element is initially displayed, it will be hidden; if hidden, it will be displayed (i.e. toggles the visibility).
- Similarly, you can specify the duration parameter for the toggle() method to make it animated like the show() and hide() methods.

```
<script>
$(document).ready(function(){
    // Toggles paragraphs display
    $(".toggle-btn").click(function(){
        $("p").toggle();
    });
});
</script>
```

```
<script>
$(document).ready(function(){
    // Toggles paragraphs with different speeds
    $(".toggle-btn").click(function(){
        $("p.normal").toggle();
        $("p.fast").toggle("fast");
        $("p.slow").toggle("slow");
        $("p.very-fast").toggle(50);
        $("p.very-slow").toggle(2000);
    });
});
</script>
```

jQuery toggle()

- Similarly, you can also specify a callback function for the toggle() method.

```
<script>
$(document).ready(function(){
    // Display alert message after toggling paragraphs
    $(".toggle-btn").click(function(){
        $("p").toggle(1000, function(){
            // Code to be executed
            alert("The toggle effect is completed.");
        });
    });
});
</script>
```

jQuery fadeIn() and fadeOut()

- You can use the jQuery fadeIn() and fadeOut() methods to display or hide the HTML elements by gradually increasing or decreasing their opacity.

```
<script>
$(document).ready(function(){
    // Fading out displayed paragraphs
    $(".out-btn").click(function(){
        $("p").fadeOut();
    });

    // Fading in hidden paragraphs
    $(".in-btn").click(function(){
        $("p").fadeIn();
    });
});
</script>
```

jQuery fadeIn() and fadeOut()

- Like other jQuery effects methods, you can optionally specify the duration or speed parameter for the fadeIn() and fadeOut() methods to control how long the fading animation will run.

- Durations can be specified either using one of the predefined string 'slow' or 'fast', or in a number of milliseconds; higher values indicate slower animations.

```
<script>
$(document).ready(function(){
    // Fading out displayed paragraphs with different speeds
    $(".out-btn").click(function(){
        $("p.normal").fadeOut();
        $("p.fast").fadeOut("fast");
        $("p.slow").fadeOut("slow");
        $("p.very-fast").fadeOut(50);
        $("p.very-slow").fadeOut(2000);
    });

    // Fading in hidden paragraphs with different speeds
    $(".in-btn").click(function(){
        $("p.normal").fadeIn();
        $("p.fast").fadeIn("fast");
        $("p.slow").fadeIn("slow");
        $("p.very-fast").fadeIn(50);
        $("p.very-slow").fadeIn(2000);
    });
});
</script>
```

jQuery fadeIn() and fadeOut()

- You can also specify a callback function to be executed after the fadeIn() or fadeOut() method completes.

```
<script>
$(document).ready(function(){
    // Display alert message after fading out paragraphs
    $(".out-btn").click(function(){
        $("p").fadeOut("slow", function(){
            // Code to be executed
            alert("The fade-out effect is completed.");
        });
    });

    // Display alert message after fading in paragraphs
    $(".in-btn").click(function(){
        $("p").fadeIn("slow", function(){
            // Code to be executed
            alert("The fade-in effect is completed.");
        });
    });
});
</script>
```

jQuery fadeToggle()

- The jQuery fadeToggle() method display or hide the selected elements by animating their opacity in such a way that if the element is initially displayed.
- It will be fade out; if hidden, it will be fade in (i.e. toggles the fading effect).

```
<script>
$(document).ready(function(){
    // Toggles paragraphs display with fading
    $(".toggle-btn").click(function(){
        $("p").fadeToggle();
    });
});
</script>
```

jQuery fadeToggle()

- Similarly, you can specify the duration parameter for the fadeToggle() method like fadeIn()/fadeOut() method to control the duration or speed of the fade toggle animation.

```
<script>
$(document).ready(function(){
    // Fade Toggles paragraphs with different speeds
    $(".toggle-btn").click(function(){
        $("p.normal").fadeToggle();
        $("p.fast").fadeToggle("fast");
        $("p.slow").fadeToggle("slow");
        $("p.very-fast").fadeToggle(50);
        $("p.very-slow").fadeToggle(2000);
    });
});
</script>
```

jQuery fadeToggle()

- Similarly, you can also specify a callback function for the fadeToggle() method.

```
<script>
$(document).ready(function(){
    // Display alert message after fade toggling paragraphs
    $(".toggle-btn").click(function(){
        $("p").fadeToggle(1000, function(){
            // Code to be executed
            alert("The fade-toggle effect is completed.");
        });
    });
});
</script>
```

jQuery fadeTo()

- The jQuery fadeTo() method is similar to the .fadeIn() method, but unlike .fadeIn() the fadeTo() method lets you fade in the elements to a certain opacity level.

```
$(selector).fadeTo(speed, opacity, callback);
```

- The required opacity parameter specifies the final opacity of the target elements that can be a number between 0 and 1.

- The duration or speed parameter is also required for this method that specifies the duration of the fade to animation.

```
<script>
$(document).ready(function(){
    // Fade to paragraphs with different opacity
    $(".to-btn").click(function(){
        $("p.none").fadeTo("fast", 0);
        $("p.partial").fadeTo("slow", 0.5);
        $("p.complete").fadeTo(2000, 1);
    });
});
</script>
```

jQuery slideUp() and slideDown()

- The jQuery slideUp() and slideDown() methods is used to hide or show the HTML elements by gradually decreasing or increasing their height (i.e. by sliding them up or down).

```
<script>
$(document).ready(function(){
    // Slide up displayed paragraphs
    $(".up-btn").click(function(){
        $("p").slideUp();
    });

    // Slide down hidden paragraphs
    $(".down-btn").click(function(){
        $("p").slideDown();
    });
});
</script>
```

jQuery slideUp() and slideDown()

- Like other jQuery effects methods, you can optionally specify the duration or speed parameter for the slideUp() and slideDown() methods to control how long the slide animation will run.

- Durations can be specified either using one of the predefined string 'slow' or 'fast', or in a number of milliseconds; higher values indicate slower animations.

```
<script>
$(document).ready(function(){
    // Sliding up displayed paragraphs with different speeds
    $(".up-btn").click(function(){
        $("p.normal").slideUp();
        $("p.fast").slideUp("fast");
        $("p.slow").slideUp("slow");
        $("p.very-fast").slideUp(50);
        $("p.very-slow").slideUp(2000);
    });

    // Sliding down hidden paragraphs with different speeds
    $(".down-btn").click(function(){
        $("p.normal").slideDown();
        $("p.fast").slideDown("fast");
        $("p.slow").slideDown("slow");
        $("p.very-fast").slideDown(50);
        $("p.very-slow").slideDown(2000);
    });
});
</script>
```

jQuery slideUp() and slideDown()

- You can also specify a callback function to be executed after the slideUp() or slideDown() method completes.

```
<script>
$(document).ready(function(){
    // Display alert message after sliding up paragraphs
    $(".up-btn").click(function(){
        $("p").slideUp("slow", function(){
            // Code to be executed
            alert("The slide-up effect is completed.");
        });
    });

    // Display alert message after sliding down paragraphs
    $(".down-btn").click(function(){
        $("p").slideDown("slow", function(){
            // Code to be executed
            alert("The slide-down effect is completed.");
        });
    });
});
</script>
```

jQuery slideToggle()

- The jQuery slideToggle() method show or hide the selected elements by animating their height in such a way that if the element is initially displayed.
- It will be slide up; if hidden, it will be slide down i.e. toggles between the slideUp() and slideDown() methods.

```
<script>
$(document).ready(function(){
    // Toggles paragraphs display with sliding
    $(".toggle-btn").click(function(){
        $("p").slideToggle();
    });
});
</script>
```

jQuery slideToggle()

- Similarly, you can specify the duration parameter for the slideToggle() method like slideUp() and slideDown() methods to control the speed of the slide toggle animation.

```
<script>
$(document).ready(function(){
    // Slide Toggles paragraphs with different speeds
    $(".toggle-btn").click(function(){
        $("p.normal").slideToggle();
        $("p.fast").slideToggle("fast");
        $("p.slow").slideToggle("slow");
        $("p.very-fast").slideToggle(50);
        $("p.very-slow").slideToggle(2000);
    });
});
</script>
```

jQuery slideToggle()

- Similarly, you can also specify a callback function for the slideToggle() method.

```
<script>
$(document).ready(function(){
    // Display alert message after slide toggling paragraphs
    $(".toggle-btn").click(function(){
        $("p").slideToggle(1000, function(){
            // Code to be executed
            alert("The slide-toggle effect is completed.");
        });
    });
});
</script>
```

jQuery animate()

- The jQuery animate() method is used to create custom animations.
- The animate() method is typically used to animate numeric CSS properties, for example, width, height, margin, padding, opacity, top, left, etc.
- But the non-numeric properties such as colour or background-colour cannot be animated using the basic jQuery functionality.
- Note: Not all CSS properties are animatable. In general, any CSS property that accepts values that are numbers, lengths, percentages, or colours is animatable. However, the colour animation is not support by the core jQuery library
- To manipulate and animate the colour use the jQuery colour plugin.

jQuery animate()

- The basic syntax of the jQuery animate() method can be given with:

```
$(selector).animate({ properties }, duration, callback);
```

- The parameters of the animate() method have the following meanings:
  - The required properties parameter defines the CSS properties to be animated.
  - The optional duration parameter specifies how long the animation will run. Durations can be specified either using one of the predefined string 'slow' or 'fast', or in a number of milliseconds; higher values indicate slower animations.
  - The optional callback parameter is a function to call once the animation is complete.

jQuery animate()

- Here's a simple example of the jQuery animate() method that animates an image from its original position to the right by 300 pixels on click of the button.

```
<script>
$(document).ready(function(){
    $("button").click(function(){
        $("img").animate({
            left: 300
        });
    });
});
</script>
```

- Note: All HTML elements have static position by default. Since the static element cannot be moved, so you must set the CSS position property for the element to relative, fixed, or absolute to manipulate or animate its position.

jQuery animate()

- You can also animate multiple properties of an element together at the same time using the animate() method. All the properties animated simultaneously without any delay.

```
<script>
$(document).ready(function(){
    $("button").click(function(){
        $(".box").animate({
            width: "300px",
            height: "300px",
            marginLeft: "150px",
            borderWidth: "10px",
            opacity: 0.5
        });
    });
});
</script>
```

- Note: The CSS properties names must be camel-cased when using with the animate() method, e.g. if you want to animate the font size you need to write 'fontSize' rather than 'font-size'. Similarly, write 'marginLeft' instead of 'margin-left', 'borderWidth' instead of 'border-width', and so on.

jQuery animate()

- You can also animate the multiple properties of an element one by one individually in a queue using the jQuery's chaining feature. We'll learn more about chaining in next chapter.

- The following example demonstrates a jQuery queued or chained animation, where each animation will start once the previous animation on the element has completed.

```
<script>
$(document).ready(function(){
    $("button").click(function(){
        $(".box")
            .animate({width: "300px"})
            .animate({height: "300px"})
            .animate({marginLeft: "150px"})
            .animate({borderWidth: "10px"})
            .animate({opacity: 0.5});
    });
});
</script>
```

jQuery animate()

- You can also define the relative values for the animated properties. If a value is specified with a leading += or -= prefix, then the target value is calculated by adding or subtracting the given number from the current value of the property.

```
<script>
$(document).ready(function(){
    $("button").click(function(){
        $(".box").animate({
            top: "+=50px",
            left: "+=50px",
            width: "+=50px",
            height: "+=50px"
        });
    });
});
</script>
```

jQuery animate()

- In addition to the numeric values, each property can take the strings 'show', 'hide', and 'toggle'.
- It will be very helpful in a situation when you simply want to animate the property from its current value to the initial value and vice versa.

```
<script>
$(document).ready(function(){
    $("button").click(function(){
        $(".box").animate({
            width: 'toggle'
        });
    });
});
</script>
```

jQuery stop()

- The jQuery stop() method is used to stop the jQuery animations or effects currently running on the selected elements before it completes.
- The basic syntax of the jQuery stop() method can be given with:

```
$(selector).stop(stopAll, goToEnd);
```

- The parameters in the above syntax have the following meanings:
  - The optional stopAll Boolean parameter specifies whether to remove queued animation or not. Default value is false, that means only the current animation will be stopped, rest of the animations in the queue will run afterwards.
  - The optional goToEnd Boolean parameter specifies whether to complete the current animation immediately. Default value is false.

jQuery stop()

- Here's a simple example that demonstrates the jQuery stop() method in real action in which you can start and stop the animation on click of the button.

```
<script>
$(document).ready(function(){
    // Start animation
    $(".start-btn").click(function(){
      $("img").animate({left: "+=150px"}, 2000);
    });

    // Stop running animation
    $(".stop-btn").click(function(){
      $("img").stop();
    });

    // Start animation in the opposite direction
    $(".back-btn").click(function(){
      $("img").animate({left: "-=150px"}, 2000);
    });

    // Reset to default
    $(".reset-btn").click(function(){
      $("img").animate({left: "0"}, "fast");
    });
});
</script>
```

jQuery stop()

- Here's one more example of this method in which, if you click the "Slide Toggle" button again after starting the animation but before it is completed, the animation will begin in the opposite direction immediately from the saved starting point.

```
<script>
$(document).ready(function(){
    // Kill and toggle the current sliding animation
    $(".toggle-btn").on("click", function(){
        $(".box").stop().slideToggle(1000);
    });
});
</script>
```

Smooth Hover Effect

- While creating the animated hover effect one of the common problem you may face is multiple queued animations, when you place and remove the mouse cursor rapidly.
- Because, in this situation mouseenter or mouseleave events are triggered quickly before the animation complete.
- To avoid this problem and create a nice and smooth hover effect you can add the stop(true, true) to the method chain, like this:

```
<script>
$(document).ready(function(){
    $(".box").hover(function(){
        $(this).find("img").stop(true, true).fadeOut();
    }, function(){
        $(this).find("img").stop(true, true).fadeIn();
    });
});
</script>
```

jQuery Chaining

- The jQuery provides another robust feature called method chaining that allows us to perform multiple action on the same set of elements, all within a single line of code.

- This is possible because most of the jQuery methods return a jQuery object that can be further used to call another method. Here's an example.

```
<script>
$(document).ready(function(){
    $("button").click(function(){
        $("p").animate({width: "100%"}).animate({fontSize: "46px"}).animate({borderWidth: 30});
    });
});
</script>
```

- The above example demonstrate the chaining of three animate() method. When a user click the trigger button, it expands the <p> to 100% width. Once the width change is complete the font-size is start animating and after its completion, the border animation will begin.

jQuery Chaining

- The method chaining not only helps you to keep your jQuery code concise, but it also can improve your script's performance since browser doesn't have to find the same elements multiple times to do something with them.

- You can also break a single line of code into multiple lines for greater readability.

- For example, the sequence of methods in the previous example could also be written as:

```
<script>
$(document).ready(function(){
    $("button").click(function(){
        $("p")
            .animate({width: "100%"})
            .animate({fontSize: "46px"})
            .animate({borderWidth: 30});
    });
});
</script>
```

jQuery Chaining

- Some jQuery methods doesn't return the jQuery object. In general, setters i.e. methods that assign some value on a selection return a jQuery object, that allows you to continue calling jQuery methods on your selection.

- Whereas, getters return the requested value, so you can't continue to call jQuery methods on the value returned by the getter.

- A typical example of this scenario is the html() method. If no parameters are passed to it, the HTML contents of the selected element is returned instead of a jQuery object.

```
<script>
$(document).ready(function(){
    $("button").click(function(){
        // This will work
        $("h1").html("Hello World!").addClass("test");

        // This will NOT work
        $("p").html().addClass("test");
    });
});
</script>
```

jQuery Callback

- JavaScript statements are executed line by line. But, since jQuery effect takes some time to finish the next line code may execute while the previous effect is still running.
- To prevent this from happening jQuery provides a callback function for each effect method.
- A callback function is a function that is executed once the effect is complete. The callback function is passed as an argument to the effect methods and they typically appear as the last argument of the method.
- For example, the basic syntax of the jQuery slideToggle() effect method with a callback function can be given with:

```
$(selector).slideToggle(duration, callback);
```

jQuery Callback

- Consider the following example in which we've placed the slideToggle() and alert() statements next to each other.

- If you try this code the alert will be displayed immediately once you click the trigger button without waiting for slide toggle effect to complete.

```
<script>
$(document).ready(function(){
    $("button").click(function(){
        $("p").slideToggle("slow");
        alert("The slide toggle effect has completed.");
    });
});
</script>
```

jQuery Callback

- And, here's the modified version of the pevious example in which we've placed the alert() statement inside a callback function for the slideToggle() method.

- If you try this code the alert message will be displayed once the slide toggle effect has completed.

```
<script>
$(document).ready(function(){
    $("button").click(function(){
        $("p").slideToggle("slow", function(){
            // Code to be executed once effect is complete
            alert("The slide toggle effect has completed.");
        });
    });
});
</script>
```

jQuery Callback

- Similarly, you can define the callback functions for the other jQuery effect methods, like show(), hide(), fadeIn(), fadeOut(), animate(), etc.

- Note: If the effect method is applied to multiple elements, then the callback function is executed once for each selected element, not once for all.

```
<script>
$(document).ready(function(){
    $("button").click(function(){
        $("h1, p").slideToggle("slow", function(){
            // Code to be executed once effect is complete
            alert("The slide toggle effect has completed.");
        });
    });
});
</script>
```

- If you try the above example code, it will display the same alert message two times once per <h1> and <p> element, upon clicking the trigger button.